

A Model-Driven Approach for Event-Based Business Process Monitoring

Falko Koetter¹ and Monika Kochanowski²

¹ University of Stuttgart IAT, Germany

`falko.koetter@iao.fraunhofer.de`

² Fraunhofer IAO, Germany

`monika.kochanowski@iao.fraunhofer.de`

Abstract. Today event-driven business process management has matured from a scientific vision to a realizable methodology for companies of all sizes and shapes. However, leveraging the power of complex event processing for supporting business process monitoring is cumbersome because of the complicated modeling of rules and alerts as well as key performance indicators in machine readable format using the event languages. However, using a model-driven approach for generating a monitoring infrastructure based on events like the aPro architecture is one possibility to enable companies with various infrastructures to leverage the advantages of business process monitoring. This paper describes how KPIs are modeled and transferred into event rules by a model-driven approach.

Keywords: business process management, process monitoring, business process goals, complex event processing, business intelligence.

1 Introduction

Today many companies work at improving their business processes and spend lots of time and effort on this [8]. However apart from the business process management strategies like CMMI [8] or the concepts of automated workflow execution [19][3], the transparency and monitoring [3] of the business processes is a major point for reacting fast on market changes [7]. One method for leveraging transparency is the introduction of complex event processing [13] (CEP) techniques in service processes [18][5]. CEP allows extracting information from business processes based on events for example from distributed service systems.

However, creating a CEP infrastructure is a cumbersome task. Apart from setting up the CEP engine, the business rules in business processes need to be transferred into an event language, the monitoring infrastructure including event collection has to be created, and the events need to be generated within the business processes or legacy systems supporting the business processes. In order to simplify these steps various approaches are possible. For example, one could use a graphical modeling language for simplifying the rule generation [18][5]. However, this only supports one task - the task of modeling rules - in the manifold

challenges. Another way is to use built-in monitoring technology in a workflow engine [3]. These approaches are usually limited to one workflow technology and are therefore not applicable for distributed systems with different system environments. Therefore, we developed a model-driven approach in order to generate the complete monitoring infrastructure in an overall methodology called architecture for business process optimization (aPro) [12], which uses a format called ProGoalML as a basis. In this paper we will extend the architecture by automatic creation of CEP rules, formal definitions of the transformation algorithm, as well as description of the challenges in creating rules for certain process patterns.

This paper is organized as follows. In Section 2 we describe the aPro architecture. Based on this, we analyze additional needs for monitoring business processes and derive new ProGoalML elements fulfilling these needs in Section 2.1. In Section 2.2, we will describe the creation of CEP rules based on ProGoalML. In Section 2.3, we will describe challenges including exclusive gateways, parallelization in business processes, as well as iteration in detail. Section 3 shows how the concept is applied to a real-world use case of support request processing. We compare our approach to related work in Section 4. Finally, in Section 5 a conclusion and an outlook on future work are given.

2 Concept

In this section we give a short overview of the aPro approach (see Figure 1) [12]. In order to monitor a process, aPro uses four components: (1) *Monitoring stubs and web services* are used to collect measurements from application systems, (2) a *CEP engine* processes these measurements and calculate key performance indicators (KPIs) and goals, the resulting data is stored in a (3) *data warehouse*. A (4) *dashboard* displays real-time as well as long-time monitoring data and alerts

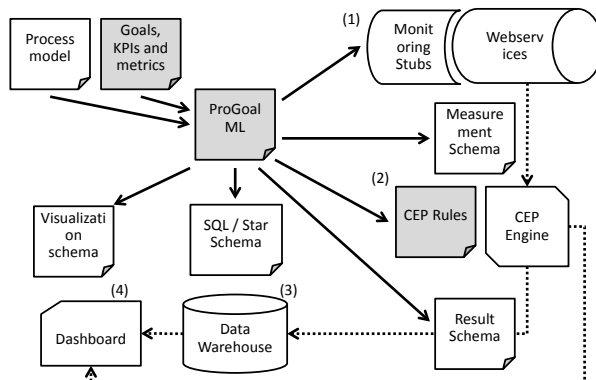


Fig. 1. Overview of documents created with ProGoalML. Dotted lines indicate flow of monitoring data between components. Grey parts highlight focus of this work.

the user in case a goal is violated. Configuration of aPro begins with modeling a process model in BPMN 2.0. In order to define what is to be monitored within the process, we extended BPMN 2.0 with modeling elements for metrics, KPIs, and goals for allowing the user to add a *goal model* to the process. Process and goal model are stored in ProGoalML, the *Process Goal Markup Language*, which serves as a central interchange format from which all other configuration files are generated. This file is used to generate the complete monitoring infrastructure automatically for standard monitoring problems and only needs human intervention for complex cases as well as additional requirements, like fine-tuning the dashboard.

2.1 Extended aPro Notation and Additional ProGoalML Elements

The aPro notation has been introduced in [11]. However, for allowing a purely model-driven approach for event-driven business process monitoring, it has to be possible to model two novel elements: (1) *aggregated KPIs*, which allow KPIs to be filtered aggregated over certain time periods, and (2) *timing goals*, which allow alerting the user based on non-conducted measurements (missing event). An overview of all elements is shown in Figure 2.

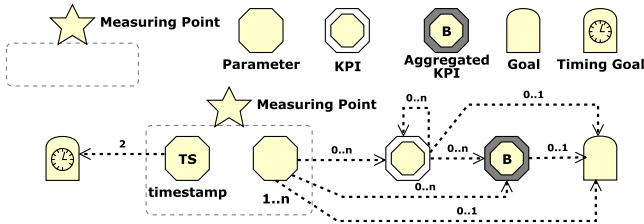


Fig. 2. ProGoalML modeling elements

A single measured value is called a *Parameter*; these are grouped in *measuring points*. A measuring point may be attached to any BPMN element at which a measurement shall occur. Whenever this element is reached during execution, the values of all parameters of the attached measuring point are included in a data set called a *measurement*.

A single *Parameter* p is a tuple

$$p \in P = (n, t, E_p)$$

where P is the set of all parameters, n is the name and $t \in T$ is the type. If $t = \text{Enumeration}$, then E_p is the set of possible values of p , else $E_p = \emptyset$.

$T = \{\text{Boolean, Enumeration, Integer, Double, String, ID, Long, Timestamp}\}$

is the set of types, determining the possible values. The type *Timestamp* is used for measuring points in time. As each measurement occurs at a specific

time, each measuring point has to have a parameter *timestamp* of type *Timestamp*, which is either modeled explicitly or added to the generated ProGoalML automatically.

A *measuring point* m is defined as a tuple

$$m \in M = (e, P_m)$$

where M is the set of all measuring points, e is the BPMN element the measuring point is attached to and P_m is the set of parameters belonging to m . A parameter may only belong to a single measuring point. However, names of parameters have to be unique only within the measuring point:

$$\forall p_1, p_2 \in P : (\exists m \in M : p_1, p_2 \in P_m) \rightarrow n_{p_1} \neq n_{p_2}$$

Each measuring point has to contain the timestamp of the measurement:

$$\forall m \in M : ('timestamp', Timestamp, \emptyset) \in P_m$$

A *Key Performance Indicators* (KPI) is seen "as a key process metric, which is evaluated in a certain analysis period and has a target value which should be reached or preserved within the analysis period indicating the achievement of predefined business goals" [20] and in our approach is calculated from the gathered measurements, and the target value to be reached is called *goal* here (see below). In aPro two kinds of KPIs are distinguished: (1) a *regular KPI* is calculated once for each process instance from the measurements and other KPIs of the process instance and is defined as follows:

$$k \in K = (n, f, V_k)$$

where K is the set of all regular KPIs, n is the Name of the KPI, f is the function to calculate the KPI and $V_k \subset P \cup K$ is the set of input variables.

In comparison, an (2) *aggregated KPI* is calculated from the measurements of multiple process instances, e.g. to get an average:

$$a \in A = (n, f, V_k, w, c)$$

where A is the set of all aggregated KPIs, and n is the name. f is the aggregating function like average, sum, count, etc. w is the window used for aggregation. It may either specify a length (number of measurements) or a time window (measurements in a timespan). c is a filter condition for measurements to be used in the aggregation (e.g. successful process instances). Aggregated KPIs may not serve as input variables for other KPIs as their windows might be different and correct calculations might not be possible.

Goals define desired results of the process. A goal is defined as follows:

$$g \in G = (n, f, v)$$

where G is the set of all goals, f is the function to determine if the goal is fulfilled and $v \in P \cup K \cup A$ is the input variable. Goals are calculated from a single input. A special goal type is the *timing goal* tg , which imposes a timing restriction between two measurements and is defined as follows:

$$tg \in TG = (t, p_s, p_e)$$

with TG as the set of all timing goals, p_s as the parameter timestamp of the first measurement, p_e as the parameter timestamp of the second measurement and t as the maximum acceptable time between both measurements. The timing goal is fulfilled iff $p_e - p_s \leq t$. Differentiating timing goals from regular

goals is necessary in order to detect a timing violation in absence of the second measurement as soon as more time than t has passed.

2.2 Generic Approach for Rule Generation

For model-driven automated monitoring *CEP rules* have to be created automatically from the ProGoalML, providing following functionality:

1. Correlate all measurements of a single process instance
2. Calculate regular KPIs and goals after the instance has finished
3. Calculate aggregated KPIs and goals every time an instance has finished
4. Raise an alert if a goal or timing goal is violated

On 1.: Parameters of type *ID* are used to identify process instances and thus to correlate measurements within instances. If two measurements contain an ID parameter with the same name and the same value, both measurements are assumed to stem from the same process instance and can be correlated. In case of multiple IDs within one instance it might be necessary to match these similarly to a join in a relational database [11]. If measurements within one instance cannot be correlated with each other, the process is divided in multiple *coverage classes* cc , for each of which a separate rule set is generated:

$$\begin{aligned}
 cc &\in CC = (M_{cc}, K_{cc}, A_{cc}, G_{cc}) \\
 M_{cc} &= \{m \in M \mid \forall m_a \in M_{cc} \exists m_0, \dots, m_n \in M : m_0 = m_a \wedge m_n = m \wedge \forall i \in [0, n) : \exists p_1 \in P_{m_i}, p_2 \in P_{m_{i+1}} : n_{p_1} = n_{p_2} \wedge t_{p_1} = t_{p_2} = ID\} \\
 K_{cc} &= \{k \in K \mid V_k \subseteq P_{cc} \cup K_{cc}\} \\
 A_{cc} &= \{a \in A \mid V_a \subseteq P_{cc} \cup K_{cc}\} \\
 G_{cc} &= \{g \in G \mid v_g \in P_{cc} \cup K_{cc} \cup A_{cc}\}
 \end{aligned}$$

with the set of parameters $P_{cc} = \{p \mid p \in P_m \wedge m \in M_{cc}\}$. Each coverage class thus contains all measurement points which may be correlated as well as all calculated KPIs and goals. If a KPI is calculated from multiple parameters, these parameters have to be in the same coverage class, because otherwise the set of input variables cannot be determined:

$$\forall k \in K : \forall v_1, v_2 \in V_k : \exists cc \in CC : v_1, v_2 \in P_{cc} \cup K_{cc}$$

The rules generated in the first step gather all measurements for each coverage class using the IDs and a pattern. All parameters p and their measured values v of a process instance are written in a single complex event e_0 :

$$e_0 = \{(p, v) \mid \exists m \in M_{cc} : p \in P_m \wedge v = \text{measurement}(p)\}$$

On 2.: The next step is to calculate the values of all regular KPIs and goals. As KPIs and goals may use other KPIs as input variables, an order of calculation has to be found. In order to calculate a KPI, all its input variables have to be available beforehand. Using the relation of input variables, we can define a graph of calculation dependencies as follows:

$$\begin{aligned}
 G_{cc} &= (V_{cc}, E_{cc}) \\
 V_{cc} &= P_{cc} \cup K_{cc} \cup \{g \mid g \in G_{cc} \wedge v_g \notin A\} \\
 E_{cc} &= \{(s, k) \mid k \in K_{cc} \wedge s \in V_k\} \cup \{(s, g) \mid g \in G_{cc} \wedge s = v_g\}
 \end{aligned}$$

Cycles may not be modeled, as calculation of KPIs in the cycle would be impossible. The dependency graph is thus a directed acyclic graph, where the

calculation order can be obtained using a modified algorithm for topological ordering [9]:

Algorithm for calculation order

```

Rest :=  $V_{cc} \setminus P_{cc}$ 
Calculated :=  $P_{cc}$ 
i := 0
while ( $Rest \neq \emptyset$ )
    i := i + 1
     $Step_i := \{k \in Rest \mid k \in K_{cc} \wedge V_k \subseteq Calculated\}$ 
     $\cup \{g \in Rest \mid g \in G_{cc} \wedge v_g \in Calculated\}$ 
     $Calculated := Calculated \cup Step_i$ 
     $Rest := Rest \setminus Step_i$ 
endwhile

```

For each $Step_i$ calculated by the algorithm a rule has to be generated which calculates the KPIs and goals in $Step_i$ using the results of the previous step $i - 1$ and adding the result to the complex event:

$$e_i = e_{i-1} \cup \{(x, v) \mid x \in Step_i \wedge v = f_x(e_{i-1})\}$$

On 3.: The result of the last step is an event e containing the values of all regular KPIs, goals, and parameters for a single process instance. These events serve as input for the calculation of aggregated KPIs, which use separate rules as they rely on a context beyond a single instance. A rule is generated for each aggregated KPI a calculating the result of its function f_a for all applicable events:

$$e_a = (n_a, f_a(\{e \mid e \in w_a \wedge c(e) = true\}))$$

Applicable events are the events which occur within the specified window at time of rule evaluation and fulfill the filter condition.

On 4.: For each goal g an alert rule is defined, which generates an alert event e_g whenever the goal is not fulfilled. Depending on the KPI or parameter the goal is defined on, the appropriate events have to be used as input variables. As goals and regular KPIs of a process are only calculated after an instance is finished, timing goals use separate rules to alert in case of missing or late measurements. For each timing goal tg a pattern is used which generates an alert event e_g if after receiving a measurement containing p_s the time t passes without receiving a measurement containing p_e .

2.3 Challenges in Rule Generation

An *event pattern* is a sequence of specific events which triggers the generation of a complex event [13]. In a sequential process, generating these patterns is straightforward as the pattern only has to account for all measurements arriving in sequential order. However, dealing with complex, non-linear process structures generating these patterns poses some challenges. This includes basically three types of problems: (a) conditional branches, (b) parallel execution of tasks, and (c) loops (see Figure 3).

For conditional branches in process models several problems appear. First of all, in scenario (a.1) in Figure 3 on the left side it is impossible to detect if

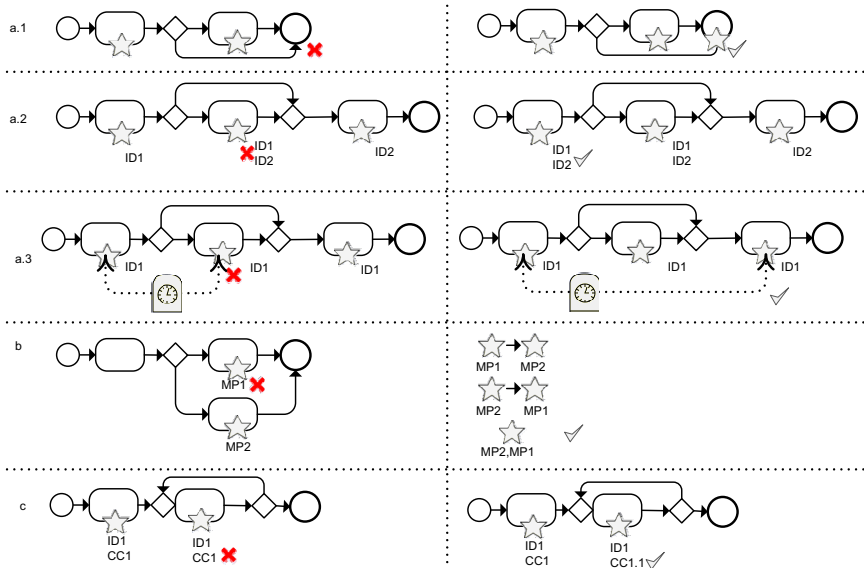


Fig. 3. Challenges in modeling goals, KPIs, and parameters in complex situations

a process has finished. Using the first measuring point marked by a star, the process start is detected, and a measurement for the second measuring point is anticipated. However, using the empty process branch, there will never be a second measurement. As this is a variant of the halting problem [17], it is undecidable. To resolve this issue, a third measuring point at the end of the process needs to be inserted (see right side).

A different scenario (a.2) in Figure 3 deals with the detection of coverage classes for measuring points [12]. On the first glance, all measuring points belong to the same coverage class. However, the second measuring point which is needed to correlate *ID1* and *ID2* is conditional and thus some process instances will not contain the measurement, making it impossible to correlate measurements and calculate KPIs. Therefore, it shall not be allowed to merge coverage classes using conditional measuring points (resolution on the right).

In scenario (a.3) it is shown that timing goals cannot be positioned in and outside of a certain conditional branch. Depending on the process instance, the second measuring point might not be traversed at all, resulting in (invalid) alerts as the measurement is never taken. If regular KPIs are calculated across conditional measurements, the KPI value is defined as *null* if one measurement with an input variable is missing; a null value is neglected for aggregated KPIs.

When dealing with parallel measurements, each measurement may be taken first (b). Thus, the pattern must be able to recognize all possible combinations.

In loop scenarios (c) it is difficult to detect how many loop executions are performed. Otherwise, similarly to (a.1) it is impossible to wait for an infinite number of loops, making it impossible to determine when the process instance

has finished. Additionally, the semantics of a measuring point within a loop are unclear considering what the results of measuring have to be. Consider for example a KPI between both measuring points in (c). If multiple measurements of the second point occur, it is unclear which is to be used as an input variable for KPI calculation. Therefore, steps within a loop are regarded as their own coverage class, which is not merged with measuring points outside the loop even if the same IDs are used. This allows for example calculations of aggregated KPIs outside the loop and inside the loop, but not merged.

These basic cases may occur multiple times or nested within each other, thus resulting in further challenges for pattern creation. Currently, our approach has two steps. In the first step the BPMN process is converted into a Petri net[6]. A petri net is chosen in order to reduce the cases to be handled in pattern detection and in order to leave open the possibility for extending aPro to other modeling languages. This Petri net is then used to find the coverage classes and a pattern for each coverage class. Currently, the scenarios (a.1), (a.2), (a.3) are excluded, whereas (b) and (c) are resolved as described.

3 Use Case and Evaluation

The use case shows a two level support process (see Figure 4). First, the support request is opened and the request is given a *RequestID*, a parameter of type ID, measured in a measuring point called *MP1*. Additionally *MP1* contains the registered timestamp of the measurement, indicated by the reserved word *timestamp*. In the first level support process, the parameter *Resolved* marks if the request has been resolved at this stage. The total cost *TotalCost* of the process is also available at *MP2*. Second level support only is called if an issue is not resolved in first level, containing basically the same parameters. Finally, the support request is closed, monitoring when and at what level the support request was resolved.

From the described parameters, various KPIs and aggregated KPIs are derived. First of all, the duration of the second level is measured as *Duration_second_level*. This is derived from $MP3.timestamp - MP2.timestamp$, aggregated over a certain time. Measuring the cost is slightly more complicated, as the cost in our example is not available at every process step, but only at *MP2* and *MP4*. This is possible by calculating the cost of the process at the end, subtracting the cost of the first level support ($MP5.TotalCost - MP2.TotalCost$), filtering the non-resolved support requests using *MP2.Resolved*. Finally we use a timing goal called *Duration_below_5_days* in order to ensure that all support requests are closed within this timespan. Additionally there is a goal for the average duration *Duration_average* to be below 2 days *Duration_average_below_2_days*. However in contrast to the timing goal, in this measurement only completed measurements are included, causing the necessity of timing goals for possible alerts of non-submitted events. Finally, the non-aggregated KPI *Resolved_Level* is measured, which shows in which level a certain issue has been resolved. Possible values therefore indicate first level, second level, and other. An exemplary subset of the generated CEP rules follows, starting with the correlation of all measurements

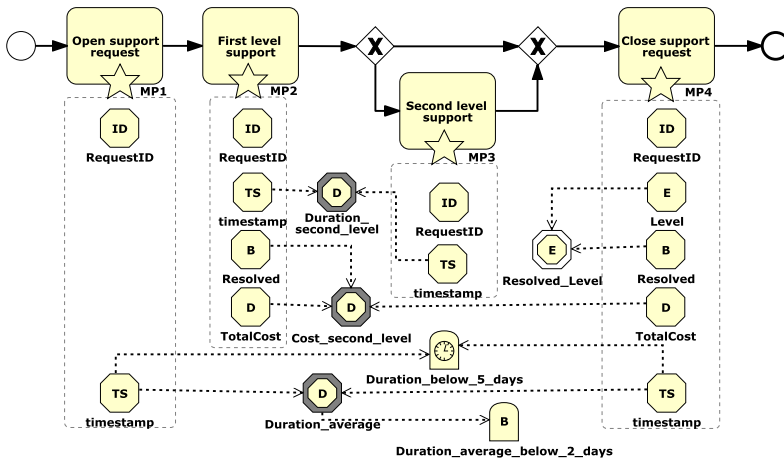


Fig. 4. Use case of two level support request processing

as described in Section 2.1 in step 1 and as described in step 2 calculating the KPIs and goals with only parameters as input variables. *IN* is the prefix used for incoming event streams for the measuring points, while *OUT* is the prefix used for outgoing event streams.

```
INSERT INTO 'OUTUseCase' SELECT MP4.RequestID AS RequestID,
MP4.Resolved AS MP4___Resolved, [...]
CASE WHEN (MP4.Level="1") THEN "1" WHEN (MP4.Resolved)
THEN "2" ELSE "Other" END AS Resolved_Level,
MP4.timestamp.after(MP1.timestamp, 0 sec, 5 day) AS
Duration_below_5_days FROM PATTERN
[(EVERY MP1 = INMP1 -> MP2 = INMP2(RequestID = MP1.RequestID) ->
MP4 = INMP4(RequestID = MP1.RequestID AND RequestID
NOT IN (select INMP3.RequestID FROM INMP3.win:keepall())) OR
(EVERY MP1 = INMP1 -> MP2 = INMP2(RequestID = MP1.RequestID) ->
MP3 = INMP3(RequestID = MP1.RequestID) ->
MP4 = INMP4(RequestID = MP1.RequestID))]
```

The calculation of aggregated KPIs follows as described in step 3. Note that the output events of the first rule are used as input for aggregated KPIs.

```
INSERT INTO 'OUTDuration_average'
SELECT (AVG(MP4___timestamp - MP1___timestamp))/3600000
AS Duration_average FROM 'OUTUseCase'.win:length(100)
```

```
INSERT INTO 'OUTDuration_average_below_2_days'
SELECT OUTDuration_average.Duration_average<(2*24) AS
```

```
Duration_average_below_2_days FROM
PATTERN [EVERY OUTDuration_average = OUTDuration_average]
```

```
INSERT INTO 'OUTCost_second_level'
SELECT AVG(MP4___TotalCost - MP2___TotalCost) AS
Cost_second_level FROM
'OUTUseCase'.win:length(100) WHERE NOT(MP2___Resolved)
```

Finally an example of an alert rule as described in step 4 is shown.

```
INSERT INTO 'OUTDuration_below_5_daysAlerts'
SELECT MP1.RequestID AS RequestID FROM PATTERN
[EVERY MP1 = INMP1 -> (timer:interval(5 day) AND NOT
MP4 = INMP4(RequestID = MP1.RequestID))]
```

To evaluate the approach, a prototype has been created which generates the above rules automatically. Additionally, an automatic deployment of the rules has been implemented in the open source CEP engine Esper [1]. The modeling tool is based on an open modeling framework, whereas the monitoring web services are based on Java REST-calls. Current work aims at implementing a dashboard automatically using a semantic dashboard description language [10].

4 Related Work

The related work to this paper is structured in two main topics: (1) approaches to modeling KPIs and (2) approaches for modeling CEP rules for monitoring.

For modeling KPIs a semantic approach is presented in [15]. Although most constructs of ProGoalML are described similarly, to our knowledge the approach does not tackle the generation of CEP rules based on a model. Another way to model metrics in a BPEL process is presented in [19]. These are then transformed for the use in Business Activity Monitoring. In [2] not only instance, but also aggregate performance metrics are considered. However, these approaches have not been used for defining CEP rules for event-driven platform-independent business process monitoring. A model-driven approach for KPIs in a BPEL process is [14], in which a monitoring model is derived across multiple abstraction levels. The complexity of the underlying platform is hidden as in aPro, but only specific systems can be monitored.

For modeling CEP rules (2) several approaches exist as well. One possibility to define complex events in a BPMN process is called BEMN [5]. However, this focuses on defining CEP rules instead of showing the business perspective of KPIs and goals. Business users are more used to thinking in KPIs, aggregating information, and specifying views on information, than in event processing language constructs. For finding service level violations [4] defines hierarchical organized Service Level Objects. Monitoring data is correlated using event logs. However, the approach we present here focuses on different event sources, not only considering logs, and the event organization is directly extracted from the

ProGoalML. In [16] events are polled from the process execution. For preparation, a process model and event descriptions and requirements are described to configure the non-intrusive monitoring. However, the creation of a monitoring policy is not performed automatically. WS-BPEL processes are validated in run-time in [3]. Similarly to ProGoalML goals compliance is checked. However, the rules are not automatically generated from a graphical or xml notation. The same accounts for [2], which extends the runtime engine, submitting events for monitoring. The rules for monitors have also to be specified manually, requiring a process engine. Similarly to CEP rules[13] they deliver numeric or Boolean values, but have to be specified separately from the process.

5 Conclusion and Outlook

In this paper we showed how the aPro approach can be used to automatically generate CEP based on the ProGoalML format. This allows for a model-driven generation of the complete monitoring infrastructure, which we have implemented in a prototype. This is a large simplification for setting up monitoring in non-workflow environments distributed over several legacy systems, allowing the involvement of business users in the modeling of their monitoring needs and allowing changes by using the model-driven approach.

However, the shortcomings of this approach include a limited complexity of possible rule patterns. Although it is possible to aggregate KPIs, it is not possible to correlate aggregated KPIs to each other, like for example having a distribution over averages over a certain enumeration, which exceeds the power of the graphical representation. In the future we will show how to integrate fully automated dashboard generation for displaying the monitored data in a user-centric way based on [10]. Finally additional work on the topics of patterns for conditional branches and loops is planned.

Altogether this paper shows a large step towards the vision of event-based model-driven business process monitoring. However, in context of the larger aPro architecture and the goal of event-driven BPM, generated rules need to provide KPIs and goals as soon as possible so adaptation is possible within the same process instance. We will further tackle this challenge when detailing adaptation.

References

1. Esper - Complex Event Processing, <http://esper.codehaus.org/>
2. Barbon, F., Traverso, P., Pistore, M., Trainotti, M.: Run-Time Monitoring of Instances and Classes of Web Service Compositions. In: International Conference on Web Services, ICWS 2006, pp. 63–71 (2006)
3. Baresi, L., Guinea, S.: Towards Dynamic Monitoring of WS-BPEL Processes. In: Benatallah, B., Casati, F., Traverso, P. (eds.) ICSOC 2005. LNCS, vol. 3826, pp. 269–282. Springer, Heidelberg (2005)

4. Bodestaff, L., Wombacher, A., Reichert, M., Jaeger, M.C.: Monitoring Dependencies for SLAs: The MoDe4SLA Approach. In: IEEE 5th Int'l. Conference on Services Computing, pp. 21–29. IEEE Computer Society Press (2008)
5. Decker, G., Grosskopf, A., Barros, A.: A Graphical Notation for Modeling Complex Events in Business Processes. In: 11th IEEE International Enterprise Distributed Object Computing Conference, EDOC 2007, p. 27 (2007)
6. Dijkman, R.M., Dumas, M., Ouyang, C.: Formal Semantics and Analysis of BPMN Process Models using Petri Nets
7. Gartner: Gartner Reveals Five Business Process Management Predictions for 2010, and Beyond (2010), <http://www.gartner.com/it/page.jsp?id=1278415>
8. Goldenson, D., Gibson, D.: Demonstrating the Impact and Benefits of CMMI: an Update and Preliminary Results. Tech. rep. (2003)
9. Kahn, A.B.: Topological Sorting of Large Networks. *Commun. ACM* 5, 558–562 (1962)
10. Kintz, M.: A Semantic Dashboard Description Language for a Process-oriented Dashboard Design Methodology. In: Proceedings of 2nd International Workshop on Model-based Interactive Ubiquitous Systems (MODIQUITOUS 2012), Copenhagen, Denmark (to appear, 2012)
11. Koetter, F., Kochanowski, M.: Goal-Oriented Model-Driven Business Process Monitoring Using ProGoalML. In: Abramowicz, W., Kriksciuniene, D., Sakalauskas, V. (eds.) BIS 2012. LNBIP, vol. 117, pp. 72–83. Springer, Heidelberg (2012)
12. Koetter, F., Weisbecker, A., Renner, T.: Business Process Optimization in Cross-Company Service Networks - Architecture and Maturity Model. In: Proceedings of the 2012 Annual SRII Global Conference (SRII 2012) (to be published, 2012)
13. Luckham, D.C.: The Power of Events: An Introduction to Complex Event Processing in Distributed Enterprise Systems. Addison-Wesley, Boston (2001)
14. Momm, C., Gebhart, M., Abeck, S.: A Model-Driven Approach for Monitoring Business Performance in Web Service Compositions. In: Proceedings of the 2009 Fourth International Conference on Internet and Web Applications and Services, pp. 343–350. IEEE Computer Society, Washington, DC (2009)
15. del-Río-Ortega, A., Resinas, M., Ruiz-Cortés, A.: Defining Process Performance Indicators: An Ontological Approach. In: Meersman, R., Dillon, T.S., Herrero, P. (eds.) OTM 2010. LNCS, vol. 6426, pp. 555–572. Springer, Heidelberg (2010)
16. Spanoudakis, G.: Non Intrusive Monitoring of Service Based Systems. *International Journal of Cooperative Information Systems* 15, 325–358 (2006)
17. Turing, A.M.: On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* 42, 230–265 (1936)
18. Vidačković, K., Weiner, N., Kett, H., Renner, T.: Towards Business-Oriented Monitoring and Adaptation of Distributed Service-Based Applications from a Process Owner's Viewpoint. In: Dan, A., Gittler, F., Toumani, F. (eds.) ICSOC/Service-Wave 2009. LNCS, vol. 6275, pp. 385–394. Springer, Heidelberg (2010)
19. Wetzstein, B., Strauch, S., Leymann, F.: Measuring Performance Metrics of WS-BPEL Service Compositions. In: Fifth International Conference on Networking and Services, ICNS 2009, pp. 49–56 (2009)
20. Wetzstein, B., Ma, Z., Leymann, F.: Towards Measuring Key Performance Indicators of Semantic Business Processes. In: Abramowicz, W., Fensel, D. (eds.) BIS 2008. LNBIP, pp. 227–238. Springer, Heidelberg (2008)